

DOT SENTINEL AI

CAPSTONE PROJECT PHASE-2

Phase – II Report

Submitted by: Group 08

Group Members List

Sl. No.	Register Number	Name
1	22BCE10071	Nandini Sharma
2	22BCE10114	Aashina Saxena
3	22BCE10247	Prakhar Shukla
4	22BCE10708	Parthib Sarkar
5	22BCE10720	Siddhant Jha

***in partial fulfillment of the requirements for the degree of Bachelor of
Engineering and Technology***



VIT[®]
BHOPAL
www.vitbhopal.ac.in

VIT Bhopal University
Bhopal
Madhya Pradesh
March, 2026



VIT[®]
BHOPAL
www.vitbhopal.ac.in

Bonafide Certificate

Certified that this project report titled “Dot Sentinel AI” is the bonafide work of “22BCE10114 Aashina Saxena, 22BCE10071 Nandini Sharma, 22BCE10708 Parthib Sarkar, 22BCE10247 Prakhar Shukla, 22BCE10720 Siddhant Jha” who carried out the project work under my supervision.

This project report (DSN4095-Capstone Project Phase-II) is submitted for the Project Viva-Voce examination held on 25.03.2026.

Supervisor

TABLE OF CONTENT

Serial Number	Topic	Page Number
1.	Introduction	6
2.	EXISTING WORK / LITERATURE REVIEW	8
3.	FRONT END, BACKEND AND SYSTEM REQUIREMENT	10
4.	METHODOLOGY	13
5.	INDIVIDUAL CONTRIBUTION	22
6.	CONCLUSION	25
7.	REFERENCE & PUBLICATIONS	31

LIST OF FIGURES

Serial Number	Figure Name	Page Number
1.	Network Anomaly Detection Process	13
2.	Heat Map	15
3.	Confusion Matrix	16
4.	Logic Diagram	17
5.	Classification Report	18
6.	Multi-class Classification Report (1)	20
7.	Multi-class Classification Report (2)	20
8.	Multi-class Classification Report (3)	21
9.	Multi-class Classification Report (4)	21
10.	UI/UX - Homepage	25
11.	UI/UX - About Us	26
12.	UI/UX - Services (1)	26
13.	UI/UX - Services (2)	27
14.	UI/UX - Services (3)	28
15.	UI/UX - Output (1)	28
16.	UI/UX - Output (2)	29
17.	UI/UX - Output (3)	29

LIST OF TABLES

Serial Number	Name	Page Number
1.	Tools and Technologies Used	18

LIST OF ABBREVIATIONS

Abbreviation	Full Form
IDS	Intrusion Detection System
SOC	Security Operations Centre
AI	Artificial Intelligence
ML	Machine Learning
LLM	Large Language Model
PCAP	Packet Capture
LSTM	Long Short-Term Memory
DBSCAN	Density-Based Spatial Clustering of Applications with Noise
API	Application Programming Interface
UI/UX	User Interface / User Experience
HMR	Hot Module Replacement
GSAP	GreenSock Animation Platform
ASGI	Asynchronous Server Gateway Interface
CLI	Command Line Interface
OOM	Out Of Memory
IP	Internet Protocol
MITRE ATT&CK	MITRE Adversarial Tactics, Techniques, and Common Knowledge
DoS	Denial of Service
KDD	Knowledge Discovery in Databases
JSON	JavaScript Object Notation
URL	Uniform Resource Locator

CHAPTER 1: INTRODUCTION

1.1 Introduction

In the modern digital landscape, the rapid expansion of networked systems and internet-based services has significantly increased the risk of cyber threats and attacks. Organizations today face sophisticated and evolving security challenges, ranging from traditional malware to advanced persistent threats and zero-day exploits. As a result, ensuring robust network security has become a critical priority for maintaining data integrity, privacy, and system reliability.

Intrusion Detection Systems (IDS) play a vital role in identifying malicious activities within network traffic. However, conventional IDS approaches, primarily based on signature and rule-based techniques, struggle to detect new and unknown attack patterns. With the growing complexity of cyber threats, there is a need for more intelligent, adaptive, and scalable solutions.

The **Dot Sentinel AI** system addresses these challenges by integrating machine learning, anomaly detection, clustering techniques, and large language models into a unified Security Operations Centre (SOC) framework. The system processes raw network traffic data (PCAP files), extracts meaningful features, and applies advanced detection models such as Random Forest, LSTM, and Isolation Forest to identify both known and unknown threats. Furthermore, it leverages a multi-agent architecture and AI-driven reporting to provide human-readable insights, reducing the burden on security analysts.

By combining automated detection with intelligent interpretation and visualization, Dot Sentinel AI aims to enhance threat detection accuracy, reduce false positives, and streamline incident response in modern cybersecurity environments.

In addition to improving detection capabilities, the integration of artificial intelligence into cybersecurity systems introduces a paradigm shift from reactive defense mechanisms to proactive and intelligent threat management. By leveraging data-driven models and automated reasoning, modern systems can not only detect anomalies but also interpret their context and potential impact in real time. Dot Sentinel AI embodies this shift by combining analytical precision with interpretability, enabling faster decision-making and reducing reliance on manual analysis. This approach not only strengthens the overall security posture of organizations but also bridges the gap between complex technical outputs and actionable insights, making advanced cybersecurity solutions more accessible and effective for both experts and non-expert users.

1.2 Motivation

The motivation for this research arises from the rapidly evolving cybersecurity landscape, where the scale, sophistication, and frequency of cyber attacks continue to grow at an unprecedented rate. Traditional intrusion detection systems, largely dependent on static rules and predefined signatures, are increasingly inadequate in identifying emerging threats such as zero-day exploits and polymorphic attacks. This limitation is further compounded by the exponential growth in network traffic data, which makes manual monitoring and analysis both inefficient and error-prone. As a result, security analysts often face challenges such as alert fatigue, high false positive rates, and delayed incident response, all of which can compromise the effectiveness of organizational security.

Moreover, while machine learning has introduced significant improvements in detection capabilities, many existing solutions operate in isolation and lack proper integration, interpretability, and real-time applicability. There is also a critical gap between technical detection outputs and actionable intelligence, which makes it difficult for organizations to respond promptly and effectively. This research is motivated by the need to develop a more holistic and intelligent system that not only detects threats but also understands, organizes, and explains them in a meaningful way. By combining machine learning, anomaly detection, clustering, and AI-driven reporting within a unified framework, Dot Sentinel AI aims to reduce complexity, enhance automation, and provide a more efficient and user-centric approach to modern cybersecurity challenges.

1.3 Objective

The objective of this research is to design and implement an advanced, intelligent intrusion detection system that addresses the limitations of traditional cybersecurity approaches while adapting to the dynamic nature of modern network environments. The system aims to transform raw network traffic into structured and meaningful insights through a combination of data preprocessing, feature engineering, and machine learning techniques. By integrating supervised learning models for attack classification alongside unsupervised anomaly detection methods, the proposed framework seeks to achieve a balanced detection mechanism capable of identifying both known and previously unseen threats.

A key objective is to enhance the interpretability and usability of cybersecurity systems by incorporating a multi-agent architecture that correlates outputs from different models and reduces false positives. Additionally, the system aims to bridge the gap between complex analytical results and human understanding by utilizing a Large Language Model to generate detailed, context-aware security reports. Another important goal is to develop a scalable and efficient pipeline that can handle large volumes of network data in real time while maintaining high performance. Ultimately, the research strives to create a unified Security Operations Centre framework that not only improves detection accuracy but also supports faster response, better decision-making, and improved overall cybersecurity management.

CHAPTER 2: EXISTING WORK / LITERATURE REVIEW

2.1 Introduction

Cybersecurity is a vital concern due to the rapid expansion of networked systems. Intrusion Detection Systems (IDS) are essential for identifying malicious activities and safeguarding systems against potential threats. This field has seen the development of a wide array of methods, evolving from older rule-based systems to modern techniques leveraging machine learning and artificial intelligence. This chapter establishes the basis for the proposed system by reviewing current intrusion detection methods, outlining their strengths and limitations.

2.2 Traditional Intrusion Detection Systems

Signature-based and rule-based techniques are the conventional foundation for intrusion detection systems. These methods work by matching incoming data against a set of predefined rules or known attack patterns (signatures) to accurately identify known threats. However, a major limitation is their inability to detect novel or zero-day attacks. Furthermore, the necessary, continuous maintenance and updating of signature databases is a time-consuming process, which ultimately hinders the adaptability of these systems to the rapidly evolving landscape of cyber threats.

2.3 Machine Learning-based Intrusion Detection

Machine learning-based intrusion detection systems have been widely adopted to improve detection accuracy and adaptability. Studies such as highlight that traditional rule-based systems fail in dynamic environments, while machine learning models can learn complex patterns from network data. Common algorithms like Random Forest and Support Vector Machines have shown strong performance in classifying network traffic into normal and attack categories. However, these models rely heavily on labeled datasets such as KDD Cup 99 and its improved versions, which may not fully represent real-world network behavior. As a result, while machine learning improves detection capability, it may still struggle with unseen or evolving attack patterns.

2.4 Anomaly Detection Techniques

Anomaly detection methods focus on identifying deviations from normal behavior rather than relying on known attack signatures. Techniques such as Isolation Forest [4] are effective in detecting rare and unusual patterns in large datasets. These methods are particularly useful for identifying zero-day attacks, which are not present in training data. However, as discussed in various studies, anomaly detection systems tend to generate higher false positives because not all unusual behavior corresponds to malicious activity. This makes it challenging to directly deploy such systems in real-time SOC environments without additional filtering or correlation mechanisms.

2.5 Clustering-based Approaches in Cybersecurity

Clustering techniques are used to group similar network activities and identify hidden patterns in traffic data. The DBSCAN algorithm, a density-based clustering method, is particularly useful in detecting clusters of malicious activity without requiring labeled data. It has been applied in intrusion detection to identify coordinated attack campaigns and group related network flows. However, clustering performance depends on parameter selection, and improper tuning can lead to incorrect grouping or noise misclassification. Additionally, clustering alone does not provide direct classification of attacks, requiring integration with other detection methods.

2.6 Limitations of Existing Systems

Despite advancements in intrusion detection, several limitations persist across existing approaches. Traditional systems lack adaptability, while machine learning models depend on dataset quality and may not generalize well to real-world scenarios . Anomaly detection techniques often suffer from high false positives , and clustering methods require careful parameter tuning . Furthermore, most existing systems operate in isolation and lack integration with intelligent reporting mechanisms. These challenges highlight the need for a more unified and intelligent framework, motivating the development of the proposed multi-agent SOC system.

CHAPTER 3 : FRONT END, BACKEND AND SYSTEM REQUIREMENT

3.1 Introduction

This chapter presents the detailed technical blueprint and foundational infrastructure of the DOT Sentinel AI project. It is structured to provide a comprehensive overview of the technologies, components, and architectural choices made for the front-end interface and the back-end processing core. Furthermore, this section outlines the specific hardware and software prerequisites necessary for both local development and successful cloud deployment of the system.

3.2 Front-End Architecture

The Front-End of DOT Sentinel AI is designed to be highly interactive, responsive, and capable of rendering complex cybersecurity intelligence (such as MITRE ATT&CK heatmaps and threat explanations) in a user-friendly manner.

3.2.1 Technology Stack

- Core Framework: React.js (v19.2.0)
- Build Tool & Bundler: Vite (v7.2.2) - Chosen for extremely fast Hot Module Replacement (HMR) and optimized production builds.
- Animations & UI Dynamics: GSAP (GreenSock Animation Platform) - Used for smooth, professional micro-animations.
- Visualizations: Chart.js - For rendering statistical data and network anomaly metrics.

3.2.2 Key Components

- Interactive Dashboard: Displays overall network health and detected anomalous flows.
- Alert Detail Panels: Translates numerical anomalies into human-readable text.
- MITRE ATT&CK Heatmap: Visually maps detected threats to specific tactics and techniques based on the MITRE framework.

3.2.3 Deployment & Configuration

- Hosting: Vercel (Configured via vercel.json for SPA routing fallback).
- Integration: Configured to dynamically connect to the backend via the VITE_API_BASE_URL environment variable, with a graceful fallback to http://localhost:8000 for local development.

3.3 Back-End Infrastructure

The Back-End is the powerhouse of the application, responsible for heavily computational tasks: parsing raw network traffic, performing machine learning inference, and communicating with Large Language Models.

3.3.1 Technology Stack

- Core Framework: FastAPI - A modern, fast (high-performance) web framework for building APIs with Python 3.11+.
- ASGI Server: Uvicorn - For serving the FastAPI application asynchronously.
- Database: SQLite - A lightweight, file-based database used for persistent alert storage.

3.3.2 Data Processing & Machine Learning

- Traffic Parsing: Utilizes tools like scapy and tshark (CLI-based) to parse raw .pcap files into structured flow-level statistical features.
- Feature Engineering: Managed extensively using pandas and numpy.
- Anomaly Detection: Employs Isolation Forest (via scikit-learn). This unsupervised machine learning model was selected because it does not require labeled data, makes it highly effective for detecting unknown/zero-day attacks, and maintains a lightweight footprint.

3.3.3 Agentic AI Layer

- AI Provider: Google Generative AI (Gemini).
- Functionality: Translates statistical anomalies detected by the Isolation Forest model into natural language explanations. It contextualizes the threat, assigns MITRE ATT&CK mappings, and suggests actionable remediation steps.

3.3.4 Deployment & Configuration

- Hosting: Render (Configured via render.yaml for Blueprint infrastructure).
- Environment Variables: Requires GEMINI_API_KEY for LLM access.

3.4. System Requirements

To successfully run and deploy the DOT Sentinel AI project, the following hardware and software prerequisites must be met.

3.4.1 Development Environment (Local)

- Operating System: Cross-platform capability (Windows 10/11, macOS, Linux).

- Python: Version 3.11 (Explicitly defined in runtime.txt).
- Node.js & npm: Required for the Vite/React frontend environment (Node v18+ recommended based on React 19 requirements).
- System Dependencies:
 - Tshark / Wireshark: Must be installed on the host machine and accessible via the system PATH for the backend to automate packet parsing successfully.

3.4.2 Production Environment (Cloud)

- **Frontend (Vercel):**
 - No specific hardware requirements. Vercel's serverless edge network handles the distribution.
 - Requires configuration of the VITE_API_BASE_URL environment variable.
- **Backend (Render):**
 - Tier: Can operate efficiently on standard free/hobby tiers, though processing very large .pcap files may require scaling up memory allocations to prevent Out-Of-Memory (OOM) errors during Pandas DataFrame operations.
 - Build Command: `pip install -r requirements.txt`
 - Start Command: `uvicorn parser.main:app --host 0.0.0.0 --port $PORT`

3.4.3 Data Requirements

- Input Format: Raw network traffic must be provided in standard .pcap format.

CHAPTER 4: METHODOLOGY

4.1 Introduction

This chapter discusses the development and deployment of the new AI-based security operations centre (SOC) intrusion detection system (IDS) known as Dot Sentinel. The process used for developing the system is based around converting raw network traffic into useful information. This transformation occurs through a multi-step process consisting of preprocessing, features extraction, machine learning based detection, anomalous behaviour detection/categorization, grouping similar types of intrusions (i.e., clustering) and the use of large language models (LLMs) to provide meaning to all the data collected.

Dot Sentinel's architecture is designed to handle very large network datasets in an efficient manner so that SOC security analysts can use the information generated by the system as quickly and effectively as possible. All components of the Dot Sentinel architecture will be presented in-detail in the sections following this chapter.

4.2 Overall System Architecture

In a sequential (layered) architecture, the data flows through numerous processing steps from one stage to the next. The first stage collects raw network traffic and prepares the data for the next stages. The next stage performs supervised learning on the processed data to detect/identify/classify attacks. The third stage uses an anomaly-detection module, specifically the Isolation Forest, to further identify anomalous behaviour.

The outputs of the previous three stages are converted to structured events, which are then clustered and stored into campaigns using traditional clustering methods. Then, a Large Language Model uses the data from the campaigns to create human-readable SOC Reports.

The modular architecture allows for scalability, flexibility, and efficiency when processing network traffic data.

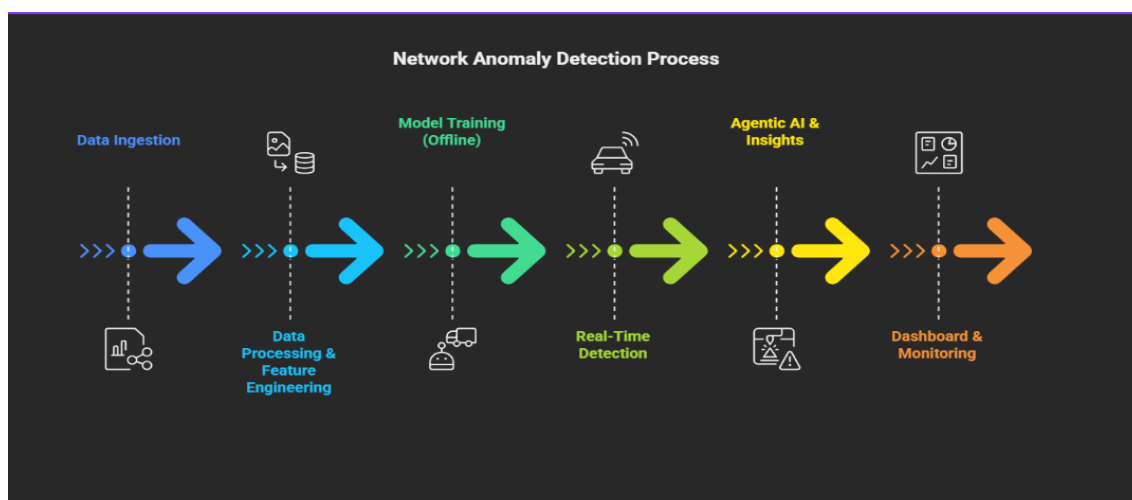


Figure 1: Network Anomaly Detection Process

4.3 Frontend Design and Implementation

The front end (interface) will allow users to have an intuitive way to see alerts, campaigns and result analysis. The interface will be developed using modern web technologies that allow the interface to be responsive (to allow users to interact easily and quickly).

The information that will be available on the interface will be structured and in a user-friendly format so users can easily see the most critical/malformed attacks, calculate the anomaly score for all fraudulent attacks and understand the campaign summaries from the data presented to them by the application.

Visualisation components will assist the users in determining the severity and type of each individual threat.

4.4 Backend Design and Implementation

The backend is tasked with managing data processing, executing models, and integrating various modules. It is built using Python and employs a modular architecture. The backend handles tasks such as data preprocessing, feature extraction, model inference, event generation, clustering, and analysis based on LLM. It facilitates seamless interaction between the frontend and machine learning components.

4.5 Database Design

The system uses structured data storage to maintain processed network data, event logs, and campaign summaries. The database design ensures efficient storage and retrieval of information required for analysis and visualization.

Data is organized in a way that supports quick access to events, anomaly scores, and campaign-level summaries.

4.6 Data Collection and Input Sources

The system uses network traffic data in the form of PCAP files as input. These files contain detailed packet-level information captured from network communication.

PCAP data provides rich information such as IP addresses, ports, protocols, timestamps, and packet sizes, which are essential for detecting malicious behavior.

4.7 Data Preprocessing

Data Preprocessing plays a crucial role in transforming raw and unstructured data into suitable format for detection and analysis. Our system receives heterogenous inputs in the form of PCAP network traffic. This form of data is noisy, inconsistent and difficult to process directly.

The initial cleaning step is to remove duplicate entries and incomplete records to insure improved data quality for analysis.

Categorical data such as protocol types and service labels is transformed into numerical form using label encoding.

To prevent any single feature from dominating, data normalization is applied to scale the feature values uniformly.

4.8 Feature Engineering

Feature engineering entails deriving significant attributes from raw network data to effectively capture traffic behavior. Attributes like flow duration, packet size, connection frequency, protocol type, and byte count are extracted. These attributes aid in differentiating between normal and malicious traffic patterns, thereby enhancing the performance of machine learning models.

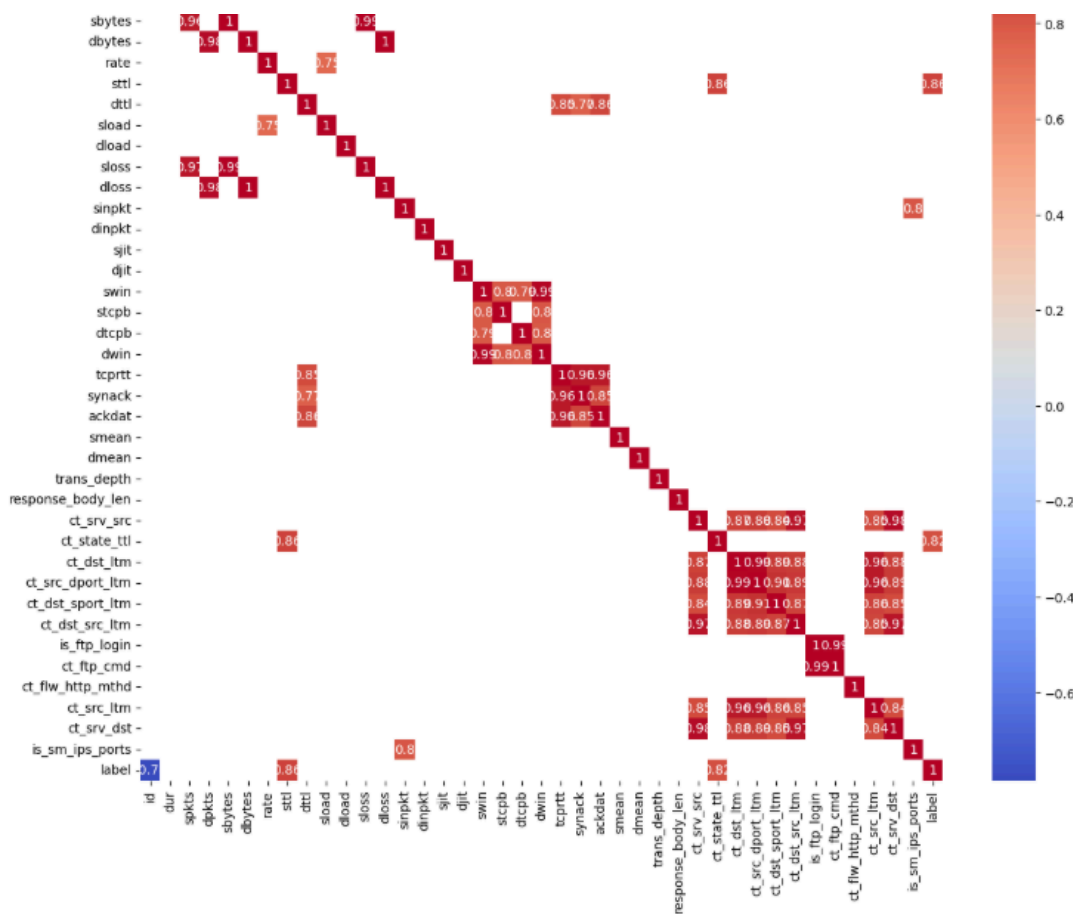


Figure 2: Heat Map

4.9 Detection System Design

The detection system consists of supervised machine learning models that classify network traffic. A binary classifier LSTM is used to determine whether a network flow is malicious or benign, while a Random Forest multi-class classifier identifies the specific type of attack.

This dual-layer detection improves accuracy and provides detailed insights into the nature of threats.

4.10 Model Development and Training

The intrusion detection system utilizes multiple models: a binary classifier for distinguishing normal from malicious traffic (yielding an attack probability), and a multi-class classifier for identifying specific attack categories (Exploits, DoS, Fuzzers, Reconnaissance, etc.). Data is preprocessed, split for training/testing, and features are selected for efficiency. An Isolation Forest model performs unsupervised anomaly detection, assigning an anomaly score to unusual network patterns. All models are evaluated using standard metrics: accuracy, precision, recall, and F1-score.

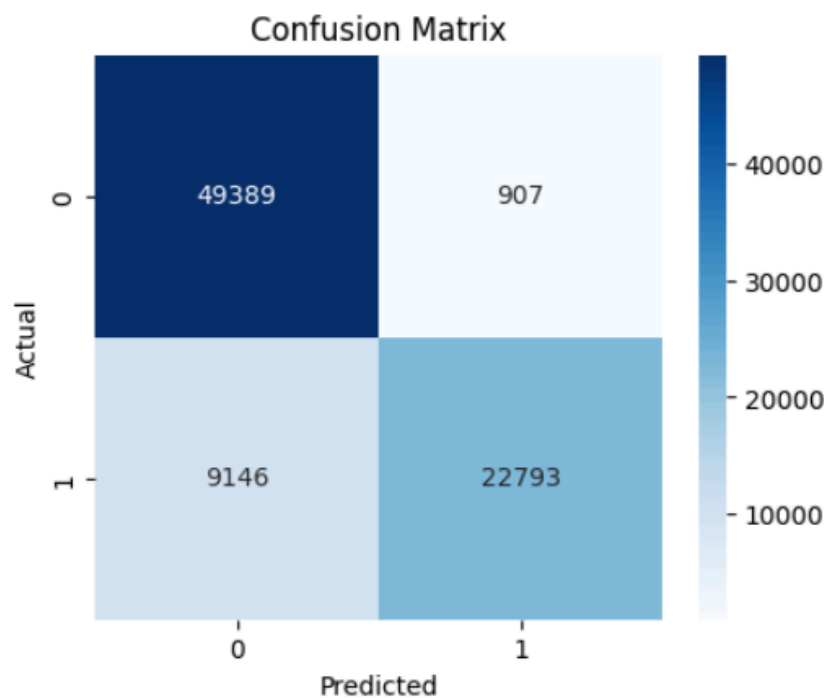


Figure 3: Confusion Matrix

4.11 Multi-Agent System Design

A modular multi-agent architecture is used to manage different stages of the SOC pipeline. A detection agent processes outputs from classification and anomaly detection models. DBSCAN is applied to group similar network flows into attack campaigns. A correlation agent combines results from multiple models to reduce false positives, while a decision agent assigns severity levels and generates insights. Additionally, a Large Language Model Gemini, with fallback support is used to generate SOC-style incident reports, including reasoning and mitigation steps. This design makes the system more flexible, efficient, and easier to interpret.

4.12 System Workflow / Working Principle

The system follows a sequential pipeline for intrusion detection, starting from data collection and preprocessing, followed by detection using classification and anomaly detection models. The outputs are further processed through clustering and correlation within a multi-agent framework. Finally, severity is assigned, and a detailed SOC-style report is generated and displayed on the dashboard.

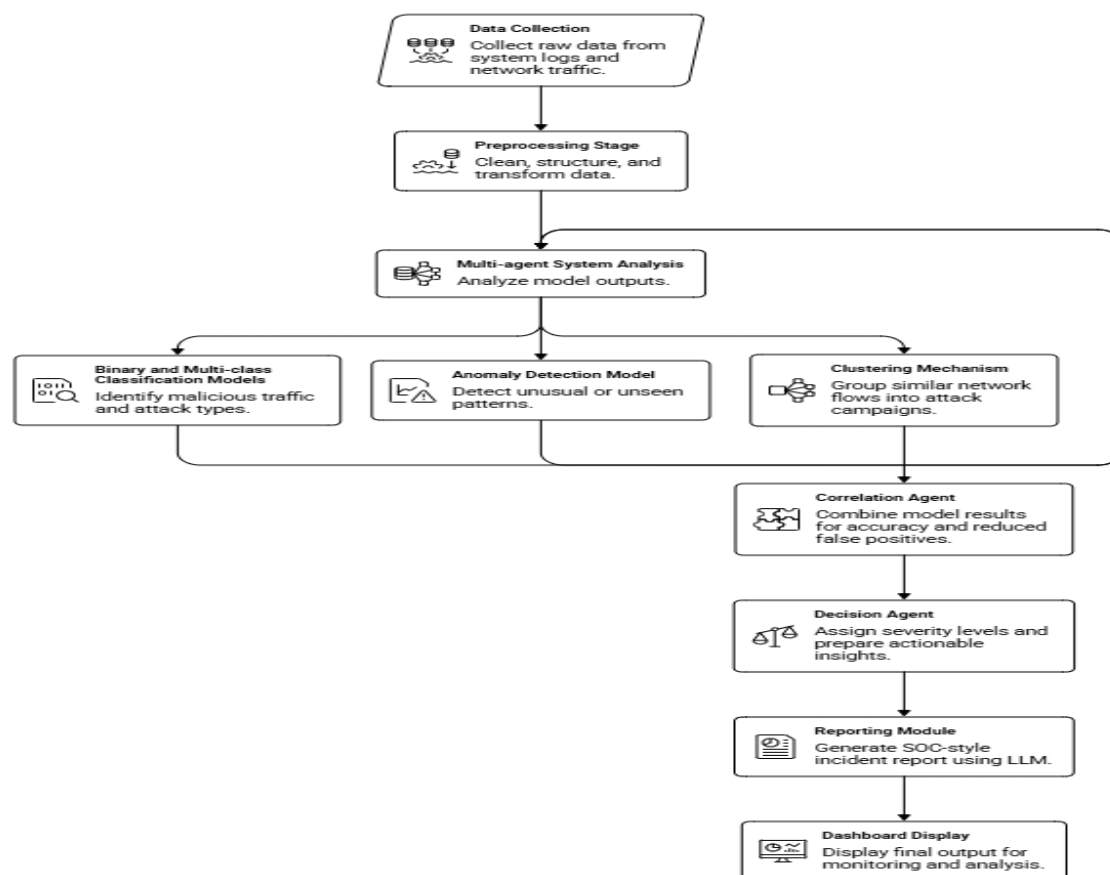


Figure 4: Logic Diagram

4.13 System Integration

The system integrates the frontend, backend, database, and detection modules into a single SOC framework. The frontend dashboard communicates with the backend through APIs to display alerts and outputs. The backend handles data processing and runs the detection models, while the database stores logs and results. The outputs from the models are further processed through the multi-agent system before being shown on the dashboard. This integration ensures smooth data flow and enables efficient and real-time monitoring of security events.

4.14 Tools and Technologies Used

The implementation of the SOC system is supported by a combination of technologies for machine learning, backend development, data handling, and system monitoring.

Category	Tools / Technologies Used
Programming Language	Python
Backend Framework	Flask
Machine Learning	Scikit-learn, Random Forest, LSTM
Data Processing	Pandas, NumPy
Clustering	DBSCAN
Anomaly Detection	Isolation Forest
Frontend	React.js (v19.2.0),Vite (v7.2.2),Tailwind CSS
LLM Integration	Gemini
Network Analysis	Wireshark
Version Control	Git, GitHub
Development Tools	VS Code

Table 1: Tools & Technologies Used

4.15 Results and Performance Evaluation

The developed system significantly enhances network anomaly detection by effectively identifying both known and novel malicious activities, surpassing traditional signature-based methods. It also clusters similar threats to aid Security Operations Center (SOC) personnel in prioritizing incident response. A key feature is the integration of a Large Language Model (LLM) for automated reporting, which processes raw network data to generate concise, digestible reports detailing the incident, detection method, and potential impact, thereby greatly streamlining incident analysis and documentation.

```
Classification Report:
      precision    recall  f1-score   support

     0       0.98      0.73      0.84      37000
     1       0.82      0.99      0.89      45332

 accuracy      0.87      82332
 macro avg      0.90      0.86      0.87      82332
weighted avg      0.89      0.87      0.87      82332

ROC-AUC: 0.9803410677619293

Binary model saved successfully.
```

Figure 5: Classification Report

```
Multiclass Classification Report:
      precision    recall  f1-score   support

 Analysis      0.03      0.02      0.02       677
 Backdoor      0.03      0.26      0.05       583
   DoS        0.32      0.21      0.26      4089
 Exploits      0.79      0.68      0.73      11132
 Fuzzers       0.79      0.74      0.77       6062
 Generic       1.00      0.97      0.98      18871
Reconnaissance  0.93      0.79      0.86      3496
 Shellcode     0.43      0.69      0.53       378
   Worms      0.71      0.45      0.56        44

 accuracy      0.76      45332
 macro avg      0.56      0.54      0.53      45332
weighted avg      0.82      0.76      0.79      45332

Multiclass model saved successfully.
```

Figure 6: Multi-Class Classification Report (1)

```
[*] Loading models and preprocessor... ...
{'flow_id': 66206, 'attack_prob': 0.355, 'attack_type': 'Fuzzers', 'anomaly_score': -0.08214895018142232}

[*] Pipeline completed successfully.
[*] Running anomaly detection...
[*] Building Event objects...
[*] Clustering campaigns...
[+] Detected 1 campaigns

{'campaign_id': 0, 'size': 1000, 'avg_attack_prob': 0.514, 'avg_anomaly_score': 0.073, 'dominant_attack_type': 'Exploits'}

[*] Sample events:
{'flow_id': 11677, 'attack_prob': 0.79, 'attack_type': 'Reconnaissance', 'anomaly_score': 0.07707531604704582}
{'flow_id': 45750, 'attack_prob': 0.815, 'attack_type': 'Reconnaissance', 'anomaly_score': 0.13433487157789453}
{'flow_id': 14719, 'attack_prob': 0.605, 'attack_type': 'Generic', 'anomaly_score': -0.05709140265706458}
{'flow_id': 54714, 'attack_prob': 0.525, 'attack_type': 'Generic', 'anomaly_score': 0.07597401095069889}
{'flow_id': 66206, 'attack_prob': 0.355, 'attack_type': 'Fuzzers', 'anomaly_score': -0.08214895018142232}
```

Figure 7: Multi-Class Classification Report (2)

```
(venv) PS C:\Users\ DELL\Desktop\soc-intrusion detection> python -m ml.pipeline
[*] LLM SOC Analysis:

=== Campaign 0 ===
Okay, here's a SOC-style incident report based on the provided campaign data.

---

**Incident Report**

**Date:** October 26, 2023
**Subject:** Potential Reconnaissance Campaign - Campaign ID 0
**Severity:** MEDIUM

**1. Incident Description**

This report details a potential reconnaissance campaign detected within our network. The campaign, identified by Campaign ID 0, has been observed across 39 flows. The primary attack type identified is reconnaissance, suggesting the attacker is actively gathering information about our systems and network infrastructure. The campaign's average attack probability is 0.72, indicating a moderate likelihood of successful exploitation. The average anomaly score of 0.074 suggests that the observed activity is below the threshold for immediate escalation, but warrants further investigation. The confidence distribution indicates a moderate level of confidence in the observed activity - 24 flows are 'MEDIUM' confident, 14 are 'LOW' confident, and 1 is 'HIGH' confident.

**2. Severity Assessment**

Based on the provided data, this incident falls into the **MEDIUM** severity category. While the anomaly score is low, the presence of 39 flows and the dominant reconnaissance type suggest a potentially significant and ongoing activity. The moderate attack probability and confidence distribution indicate that this campaign is not immediately catastrophic, but it requires attention. A sustained campaign of this nature could lead to broader compromise and potential lateral movement within our environment.

**3. Reasoning**
```

Figure 8: Multi-Class Classification Report (3)

4.16 Discussion

The results demonstrate that the proposed system is capable of effectively detecting and analyzing network intrusions using a combination of machine learning models and a multi-agent framework. The integration of classification and anomaly detection helps in identifying both known and unknown threats, while clustering provides additional insight into related attack patterns. The use of a Large Language Model improves the interpretability of results by generating detailed and understandable incident reports. However, the system's performance depends on the quality of input data and preprocessing. In some cases, complex or ambiguous patterns may still lead to false positives. Overall, the system shows promising performance and highlights the potential of combining machine learning and agent-based approaches in modern SOC environments. The proposed system successfully detects and analyzes network intrusions by integrating machine learning models within a multi-agent framework. This approach effectively identifies both known and unknown threats through a

combination of classification and anomaly detection. Furthermore, clustering analysis provides valuable insights into related attack patterns. The Large Language Model enhances the utility of the results by generating clear and detailed incident reports, thereby improving interpretability. While the system demonstrates promising performance and the potential for use in modern Security Operations Center (SOC) environments, its effectiveness is contingent upon the quality of the input data and preprocessing. A limitation is the potential for false positives when encountering overly complex or ambiguous network patterns.

CHAPTER 5: INDIVIDUAL CONTRIBUTIONS

Siddhant Jha | 22BCE10720

I was primarily responsible for designing and developing the frontend of the Dot Sentinel AI system, with a strong focus on creating a responsive, interactive, and user-friendly interface tailored for cybersecurity analysis. I implemented the dashboard using modern web technologies, enabling clear visualization of network alerts, anomaly scores, and clustered attack campaigns. My work emphasized usability and clarity, ensuring that even complex outputs from machine learning models could be easily interpreted by users. In addition to frontend development, I played a significant role in defining the overall system architecture by structuring how different modules interact and ensuring efficient data flow between components.

I also served as the leader of the group, coordinating tasks, managing timelines, and ensuring effective collaboration among all team members throughout the project lifecycle. I facilitated communication between different development areas, helping align frontend, backend, and integration efforts toward a unified goal. I worked extensively on integrating the frontend with backend APIs, enabling real-time data exchange and dynamic rendering of results. This involved handling asynchronous requests, managing state efficiently, and ensuring seamless communication between the user interface and processing layers. I also contributed to optimizing performance and maintaining consistency across different environments. Through my combined technical and leadership contributions, I ensured that the system was both well-structured and efficiently executed.

Prakhar Shukla | 22BCE10247

I was responsible for developing the backend infrastructure of the Dot Sentinel AI system, focusing on building a robust, scalable, and efficient processing pipeline capable of handling large volumes of network traffic data. My work involved implementing core functionalities such as data preprocessing, feature extraction, and transformation of raw PCAP data into structured formats suitable for analysis. I integrated multiple machine learning models, ensuring comprehensive threat detection capabilities across different scenarios.

I contributed to the architectural design of the system by organizing the backend into modular components. This modularity improved flexibility, maintainability, and scalability, allowing different components such as detection, clustering, and reporting to function independently yet cohesively. I also developed FAST APIs to enable seamless communication between the frontend and backend, ensuring efficient data exchange and real-time updates. I also ensured proper integration of data processing libraries and machine learning frameworks to maintain consistency in outputs. Through my contributions, I established a strong backend foundation that supports accurate detection, efficient processing, and smooth system operation.

Parthib Sarkar | 22BCE10708

I was responsible for the integration and deployment of the Dot Sentinel AI system, ensuring that all individual components functioned together as a unified and efficient platform. My primary focus was on integrating the frontend, backend, database, and machine learning modules to enable seamless communication and data flow across the entire system. I worked on configuring APIs, managing environment variables, and resolving compatibility issues between different technologies, ensuring that each component interacted correctly within the overall architecture. This integration was critical in transforming separate modules into a cohesive Security Operations Centre system.

In addition to integration, I handled the deployment of the application on cloud platforms, ensuring scalability, reliability, and accessibility for real-world usage. I configured hosting environments, managed build and deployment pipelines, and ensured that both frontend and backend services were properly deployed and connected. My work also involved optimizing system performance in production environments, including handling resource allocation and ensuring stability under varying workloads. I conducted testing to verify that the system functioned correctly in both local and cloud environments, addressing any issues related to deployment or runtime execution. Through my contributions, I ensured that the system was not only functional but also deployment-ready, enabling smooth operation and practical usability in real-time cybersecurity scenarios.

Nandini Sharma | 22BCE10071

I was responsible for designing and developing the core machine learning pipeline and agentic threat analysis framework for the AI-based Security Operations Center (SOC) system. My main contribution was a supervised learning-based intrusion detection mechanism, which included a binary classification model (for malicious activity probability) and a multi-class classification model (for attack categorization). Both models were trained on the UNSW-NB15 dataset. I also built a unified preprocessing pipeline, incorporating categorical feature encoding and feature normalization, to ensure reliable performance.

I designed an Event abstraction layer, where each network flow is converted into a structured event containing attributes like flow ID, attack probability, and predicted attack type. This helped in organizing raw predictions into a more meaningful and analyzable format. I also developed the campaign correlation module using the DBSCAN clustering algorithm. By applying clustering on relevant features, I grouped similar suspicious network flows into attack campaigns, which helps reduce alert fragmentation and provides a higher-level understanding of coordinated activities. I was responsible for selecting and tuning key parameters to ensure effective clustering behavior. Finally, I integrated these components into a unified end-to-end pipeline that handles preprocessing, model inference, event generation, and clustering. This ensured smooth data flow across different stages and improved the modularity of the system.

Aashina Saxena | 22BCE10114

I was responsible for developing the advanced analytical components of the Dot Sentinel AI system, with a primary focus on anomaly detection and intelligent report generation. My work involved implementing the Isolation Forest algorithm to detect abnormal and previously unseen patterns in network traffic data. I designed and integrated the anomaly detection pipeline to assign anomaly scores to each network flow, enabling the system to identify deviations from normal behavior that may not be captured by supervised models. I ensured that these anomaly scores were effectively incorporated into the overall analysis pipeline and combined with classification outputs for enhanced decision-making.

In addition to anomaly detection, I played a key role in integrating a Large Language Model (LLM) into the system using an API-based approach. I developed the logic for converting structured outputs, such as attack probabilities, attack types, and anomaly scores, into meaningful prompts that could be processed by the LLM. This enabled the generation of human-readable SOC incident reports, improving the interpretability of the system. I also ensured seamless data flow between different modules, particularly between anomaly detection, clustering, and LLM analysis, maintaining consistency and efficiency throughout the pipeline. Through my contributions, I enhanced the system's capability to not only detect anomalies but also explain them effectively, supporting faster and more informed decision-making in SOC environments.

CHAPTER 6: CONCLUSION

6.1 Introduction

This chapter presents the final outcomes of the Dot Sentinel AI project developed during Phase 2 of the Capstone. It provides a walkthrough of the fully implemented system through screenshots of each major functional module, along with brief explanations of each component's role. The chapter concludes with a summary of the overall outcomes achieved across both phases of development.

6.2 System Screenshots and Explanations

6.2.1 Landing Page / Home Interface

The landing page serves as the entry point for users. It presents the system's branding and provides navigation to the main dashboard, PCAP upload interface, and reporting modules. The interface is built with a dark theme to reduce analyst eye strain during extended monitoring sessions, implemented using React.js with Tailwind CSS.

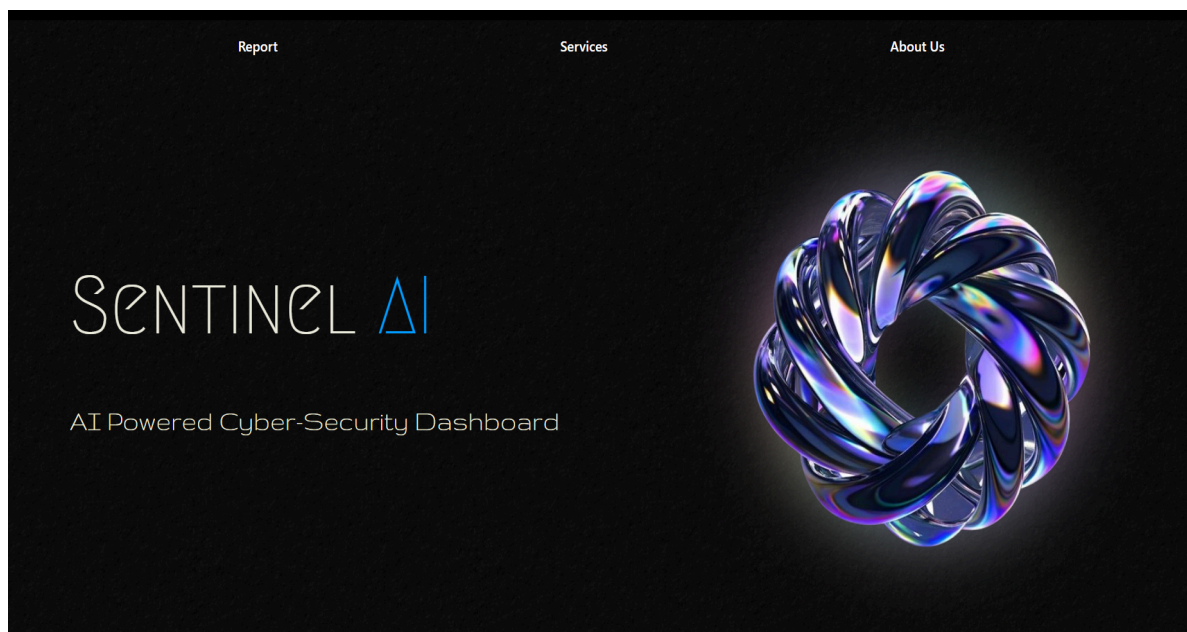


Figure 9: UI/UX - Homepage

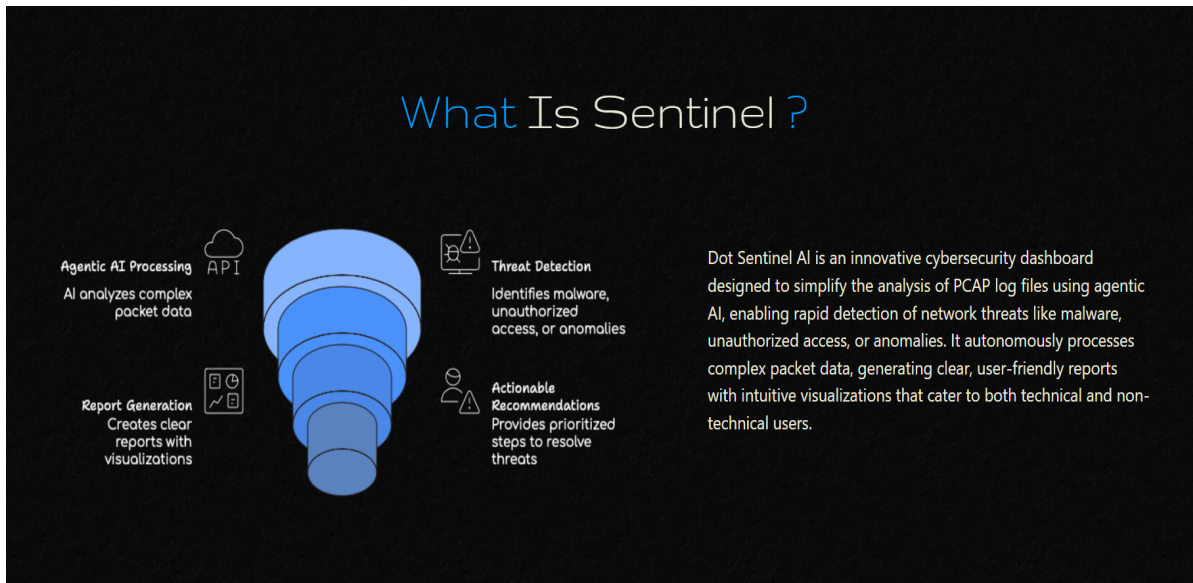


Figure 10: UI/UX - About Us

6.2.2 PCAP File Upload Interface

The PCAP upload interface allows users to submit network capture files for analysis via a simple drag-and-drop mechanism. Once uploaded, the backend immediately begins parsing — extracting packet-level features such as source and destination IPs, ports, protocols, timestamps, and flags. The system handles large files efficiently, processing up to 1 GB of PCAP data.

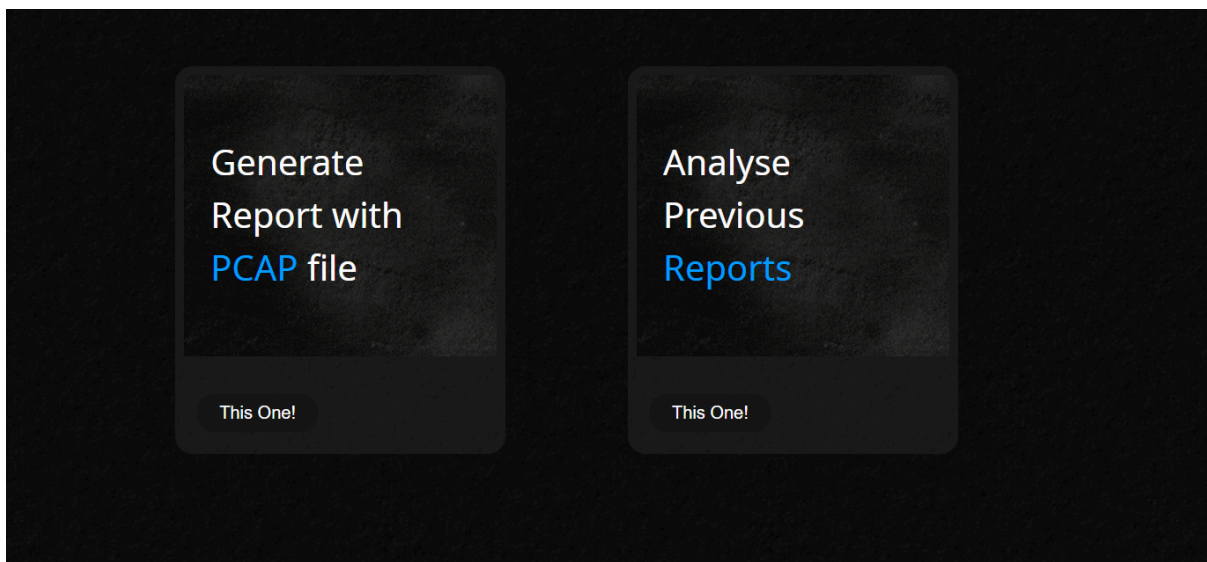


Figure 11: UI/UX - Services (1)

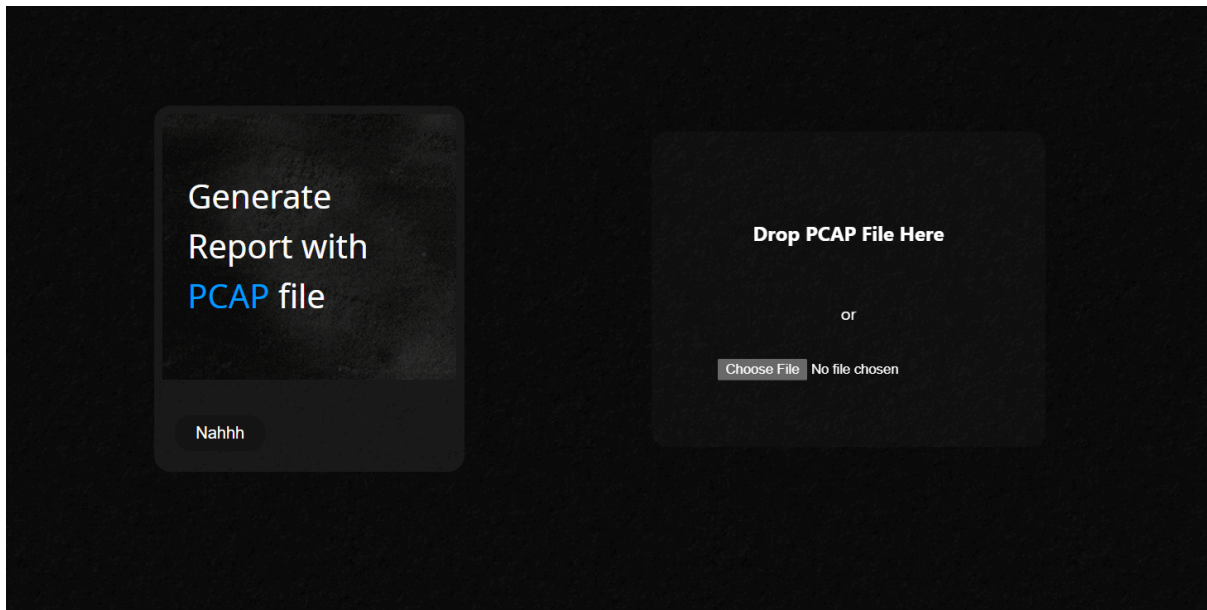


Figure 12: UI/UX - Services (2)

6.2.3 Anomaly Detection and Alert Panel

This panel displays the output of the dual-layer detection engine. Each network flow is first classified as malicious or benign by the binary classification model, followed by a multi-class classification model for attack type identification — such as DoS, Exploits, or Reconnaissance — by the Random Forest multi-class classifier. The Isolation Forest module further assigns an anomaly score to each flow. All alerts are presented in a structured table showing timestamp, source and destination IP, protocol, attack type, anomaly score, and severity level.

6.2.4 MITRE ATTACK Mapping

Detected threats are automatically mapped to the MITRE ATT&CK framework by the LLM-based MITRE Mapper agent. This screen displays a heatmap of triggered tactics and techniques, helping analysts understand not just what occurred, but how — supporting faster and more informed incident response and aiding in compliance documentation.

6.2.5 Campaign Clustering

The campaign clustering view uses DBSCAN to group related network flows into unified attack campaigns. Rather than presenting every alert individually, this view consolidates related events into coherent campaign summaries, significantly reducing alert fatigue and providing analysts with a higher-level understanding of attacker behavior across a session.

6.2.6 AI-Generated SOC Incident Report

Using the Gemma-3-1B-IT (Google) LLM, Dot Sentinel AI automatically generates structured, human-readable SOC incident reports after each analysis. Each report includes a plain-language description of the detected threat, affected hosts, MITRE ATT&CK mappings, and recommended mitigation steps — eliminating the need for manual report writing and making the system accessible to users of all technical backgrounds.

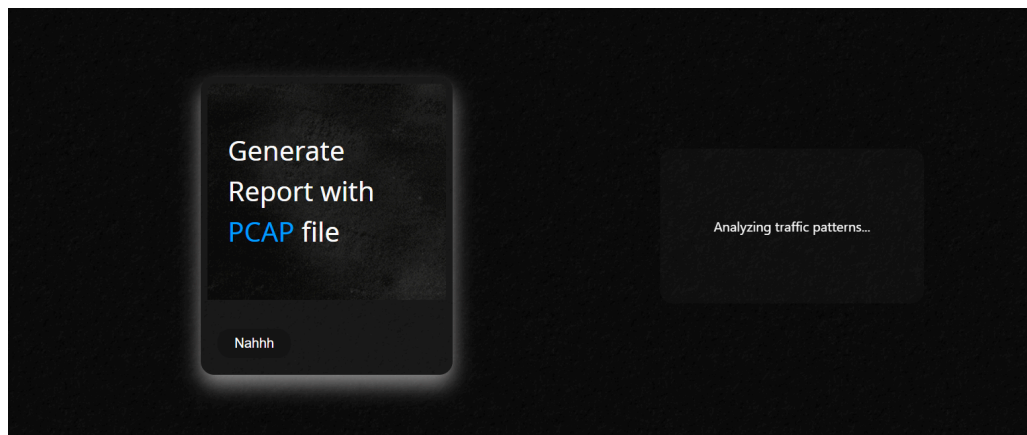


Figure 13: UI/UX - Services (3)

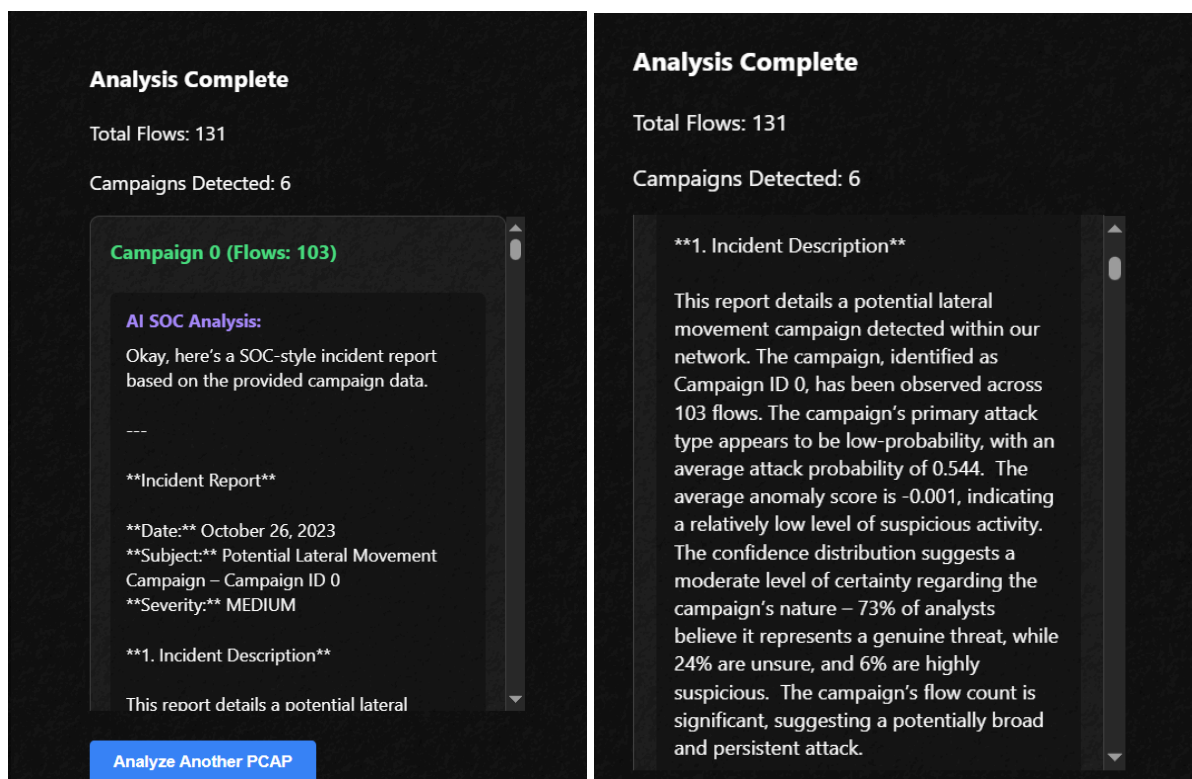


Figure 14: UI/UX - Output (1)

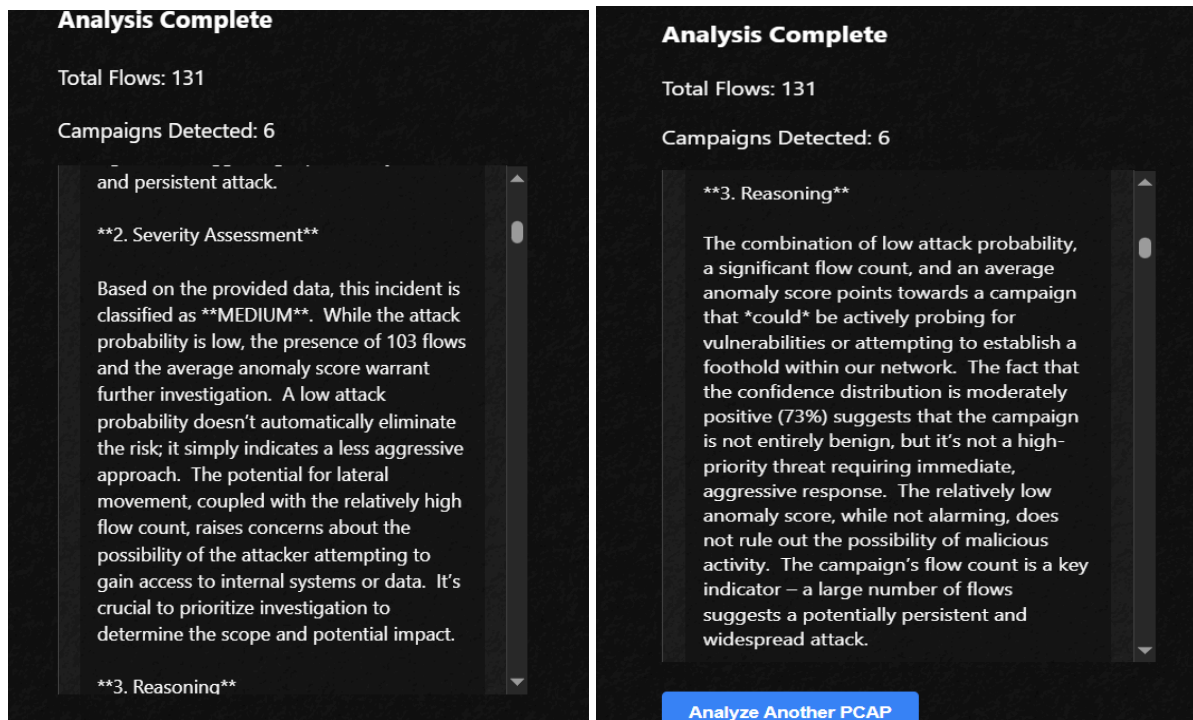


Figure 15: UI/UX - Output (2)

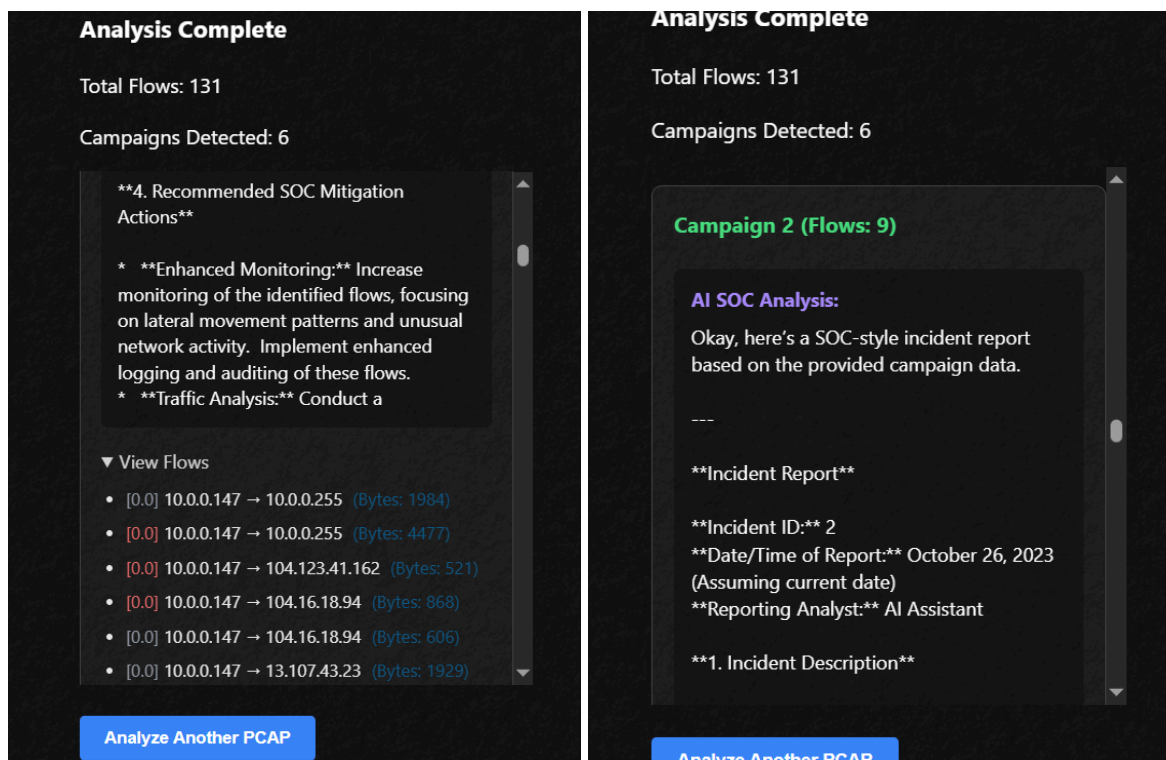


Figure 16: UI/UX - Output (3)

6.3 Final Conclusion

The Dot Sentinel AI project has successfully delivered a fully functional, AI-powered Security Operations Centre platform for automated network intrusion detection, anomaly analysis, and intelligent incident reporting, achieving all objectives set at the outset of the project.

The end-to-end PCAP processing pipeline efficiently ingests and parses large network capture files. The dual-layer detection engine — combining LSTM, Random Forest, and Isolation Forest — provides comprehensive coverage of both known and novel threat types. The multi-agent architecture powered by Gemini autonomously interprets security events, maps them to the MITRE ATT&CK framework, and generates detailed SOC reports without manual intervention. The DBSCAN campaign clustering reduces alert noise by grouping related events into meaningful attack narratives. The PCAP Generator adds a unique capability for synthetic traffic simulation, enabling testing in controlled environments. Finally, the React.js dashboard provides a clean, interactive visualization, and responsive interface accessible to both expert analysts and non-technical users.

Together, these components form a cohesive cybersecurity platform that directly addresses the core challenges of modern SOC operations — volume, expertise gaps, alert fatigue, and the need for actionable intelligence. Dot Sentinel AI demonstrates that intelligent, explainable, and accessible cybersecurity tooling is achievable through the thoughtful combination of machine learning, agentic AI, and modern software engineering, and that such tools can meaningfully contribute to a more secure digital environment for organizations of all sizes.

Reference and Publication:

1. Barenji, R. V., & Khoshgoftar, S. (2025). Agentic AI for autonomous anomaly management in complex systems. *arXiv preprint arXiv:2507.15676*.
2. Goswami, M. J. (2024). AI-based anomaly detection for real-time cybersecurity. *International Journal of Research and Review Techniques*, 3(1), 45–53.
3. Katiyar, N., Tripathi, S., Kumar, P., & Verma, S. (2024). AI and cyber-security: Enhancing threat detection and response with machine learning. *Educational Administration: Theory and Practice*, 30(4), 6273–6282.
4. Liu, F. T., Ting, K. M., and Zhou, Z.-H., “Isolation Forest,” *IEEE International Conference on Data Mining*, 2008.
5. Canadian Institute for Cybersecurity, University of New Brunswick. (2017). *CIC-IDS2017 dataset*.
6. Doshi, R., Khatri, N., Choubisa, M., & Hiran, K. K. (2022). A simple and robust approach of random forest for intrusion detection systems in cyber security. In *Proceedings of the 2022 International Conference on IoT and Blockchain Technology (ICIBT)* (pp. 1–5). IEEE.
7. Tavallaei, M. et al., “A Detailed Analysis of the KDD CUP 99 Data Set,” *IEEE Symposium on Computational Intelligence*, 2009.
8. MITRE Corporation. (2022). *MITRE ATT&CK framework documentation*.
9. Ester, M. et al., “A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases (DBSCAN),” *KDD Conference*, 1996.
10. Brown, T. B. et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
11. Ahmad, Z. et al., “Network Intrusion Detection System: A Systematic Study of Machine Learning and Deep Learning Approaches,” *Transactions on Emerging Telecommunications Technologies*, 2021.
12. Splunk Inc. (2024). *Splunk stream product documentation*.
13. *AI-driven cybersecurity framework for anomaly detection in power systems*. (2025). *Scientific Reports*.